This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Parameter-efficient Continual Learning for Enhancing Plasticity without Forgetting under Limited Model Capacity

Yitian Chen[1], Shigeng Zhang[1]*, Xuan Liu[2,3]*, Mingming Lu[1], Kai Chen[4], Hongye Zhu[2], Xinning Chen[2]

[1] Central South University, China [2] Hunan University, China [3] The Ministry of Education Key Laboratory of "Fusion Computing of Supercomputing and Artificial Intelligence", China
[4] Institute of Information Engineering, Chinese Academy of Sciences, China

Abstract

Avoiding catastrophic forgetting for previous tasks and maintaining model plasticity to support new tasks are two critical objectives of continual learning. However, existing methods usually neglect one of the two aspects and fail to support long task sequences with satisfactory performance, especially in resource-constrained scenarios in which the size of the model is limited. This work proposes GRAPA, a parameter-efficient continual learning method that well balances stability and plasticity of the model to handle long task sequences with diverse complexities. GRAPA enhances model plasticity without sacrificing stability with two novel designs. First, a gradient-guided parameter reuse strategy is proposed to make full use of frozen parameters while ensuring that no task interference is introduced. Second, a reinforcement-learning-based parameter allocation is designed to enable the model to adapt to the current task on top of reused parameters while preserving maximal model capacity for future tasks. Experiments on multiple task sequences composed of various datasets demonstrate that GRAPA lifts mean task accuracy by up to 7.67%, with up to 14.92% gains on subsequent complex tasks, reflecting GRAPA's superior plasticity.

1. Introduction

Deep learning models deployed in dynamic environments often need to learn a sequence of tasks that arrive over time. However, when trained sequentially, they usually suffer catastrophic forgetting [14], i.e. learning new tasks interferes with previously acquired knowledge and decreases the performance of old tasks. Continual learning (CL) has recently been proposed to solve this problem, which can be broadly classified into three categories: replay-based methods [15, 23], regularization-based methods [9, 11, 13, 24], and parameter isolation methods [8, 10, 12, 19]. The first two types rely on updating parameters within a shared space either by replaying old task data or by constraining changes to important parameters. In contrast, parameter isolation methods assign disjoint parameter subsets to each task, thus effectively preventing interference across tasks.

Parameter isolation methods have received increasing attention in recent years because of their superior ability in suppressing catastrophic forgetting of previous tasks. However, existing works usually neglect the preservation of the plasticity of the model that is critical to support new tasks. For example, a common strategy is to activate a distinct subset of parameters by learning a task-specific mask on a shared backbone network [19, 26]. While effectively preventing interference among tasks, this strategy limits the model's flexibility in handling task sequences with increasing complexity, for which sole parameter reuse is insufficient. To exploit the ability of new parameters, a train–prune–retrain paradigm is proposed [7, 8, 18], which allocates a fixed proportion of new parameters to each task by pruning the remaining free parameter space. However, existing works following this paradigm adopt a predefined pruning rate, ignoring diversity in task complexity and capacity requirements. As shown in Figure 1, these works may result in uneven parameter allocation and cause two potential problems. First, under resource constraints, new parameters may be exhausted before all tasks are learned, which limits the model's long-term learning capacity. Second, in task sequences of increasing complexity, early simple tasks may freeze excessive parameters, leaving insufficient capacity for subsequent, more complex tasks.

In this work, we propose a continual learning method with *gradient-guided parameter reuse and adaptive parameter allocation* (GRAPA), which aims to balance model stability and plasticity under constrained resources. For each new task, we prioritize reusing parameters from the frozen parameter space without harmful interference, and allocate

*Corresponding author.

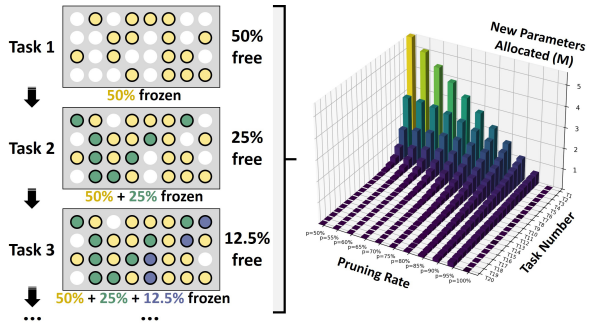


Figure 1. Left: Illustration of parameter freezing in the train–prune–retrain paradigm under a 50% pruning rate. Right: Number of frozen parameters per task when sequentially training 20 tasks on ResNet-18 with varying pruning rates.

only the minimal task-specific parameters needed to maintain performance. This enables the model to maintain high task accuracy while preserving parameter capacity for future tasks. To this end, we adopt a gradient-consistency-based reuse criterion to filter out old parameters that may introduce harmful interference to the current task, while reusing the remaining parameters without conflicts. To compensate for the performance limitations of reused parameters, we introduce an adaptive allocation method based on reinforcement learning, which achieves a good balance between improving task performance and preserving parameter space for future tasks. We validate our method on six heterogeneous task sequences across different compact models with limited capacity. These task sequences are constructed from eight diverse datasets, covering a wide range of visual domains and increasing task complexity. The results show that GRAPA consistently achieves higher accuracy and demonstrates a superior trade-off between stability and plasticity across all task sequences.

The main contributions of this work are summarized as:

- We reveal the limitations of existing parameter isolation methods in balancing stability and plasticity, especially under constrained resources and in heterogeneous task sequences with increasing complexity.
- We propose GRAPA, a continual learning framework that achieves reliable old-parameter-reuse and adaptive new-parameter-allocation, enabling interference-free learning with minimal use of additional model capacity.
- Experiments across multiple task sequences built from diverse datasets demonstrate that GRAPA inherits the stability of parameter isolation methods while significantly enhancing model plasticity on complex tasks.

2. Related Work

Overcoming catastrophic forgetting is one of the central challenges in continual learning. To address this issue, replay-based methods mitigate forgetting by reintroducing past experiences during training [2–4, 25]. However, these approaches often incur substantial storage costs and raise privacy concerns. Although using intermediate features [6, 22] or employing generative models to synthesize past data [5] offers partial relief, such strategies cannot completely eliminate this issue. Regularization-based methods address the forgetting problem by introducing constraints into the loss function. They either constrain updates to important parameters through regularization [1, 14, 29] or use knowledge distillation to preserve prior knowledge [16, 23]. Despite their conceptual differences, both replay-based and regularization-based approaches operate within a shared parameter space, where continual parameter updates across tasks inevitably limit long-term stability. In contrast, parameter isolation methods assign disjoint parameter subsets to individual tasks, offering a more robust mechanism to prevent task interference. This can be achieved by learning task-specific masks over a pre-trained [19] or randomly initialized [26] backbone network to selectively activate subsets of weights, or by jointly optimizing both the backbone and the masks during task training [10]. Other parameter isolation approaches follow a train–prune–retrain paradigm [18], enhancing adaptability by pruning redundant parameters after each task and reallocating them for future use. Some of these methods aim to promote knowledge transfer by reusing [7] or transforming [8] frozen parameters from previous tasks. However, due to the lack of efficient allocation strategies, available free parameters are quickly exhausted. As a result, such methods often fail in long task sequences, especially when task complexity increases and parameter reuse becomes less effective.

Recent works also explore continual learning with large pre-trained models [17, 27, 30], leveraging their strong representations and efficient fine-tuning techniques to achieve superior performance on complex tasks. However, their reliance on substantial computational and memory resources may limit their deployment in resource-constrained on-device scenarios. In contrast, this work focuses on continual learning under limited model capacity, aiming to enhance model plasticity without catastrophic forgetting in resource-constrained scenarios like on-device environments.

3. Problem Formulation

In continual learning with parameter isolation, a deep neural network incrementally learns a sequence of tasks $T_1, T_2, \ldots, T_N$ that arrive over time. To prevent catastrophic forgetting, the model assigns each task T_i a disjoint set of parameters ϕ_i, satisfying the isolation constraint $\phi_i \cap \phi_j = \emptyset,\ \forall i \neq j$. When training on task T_t, the model freezes the parameters $\phi_1, \phi_2, \ldots, \phi_{t-1}$ assigned to previous tasks and prevents them from being updated during backpropagation. To enhance learning efficiency by leveraging prior knowledge, task T_t is allowed to employ a sub-

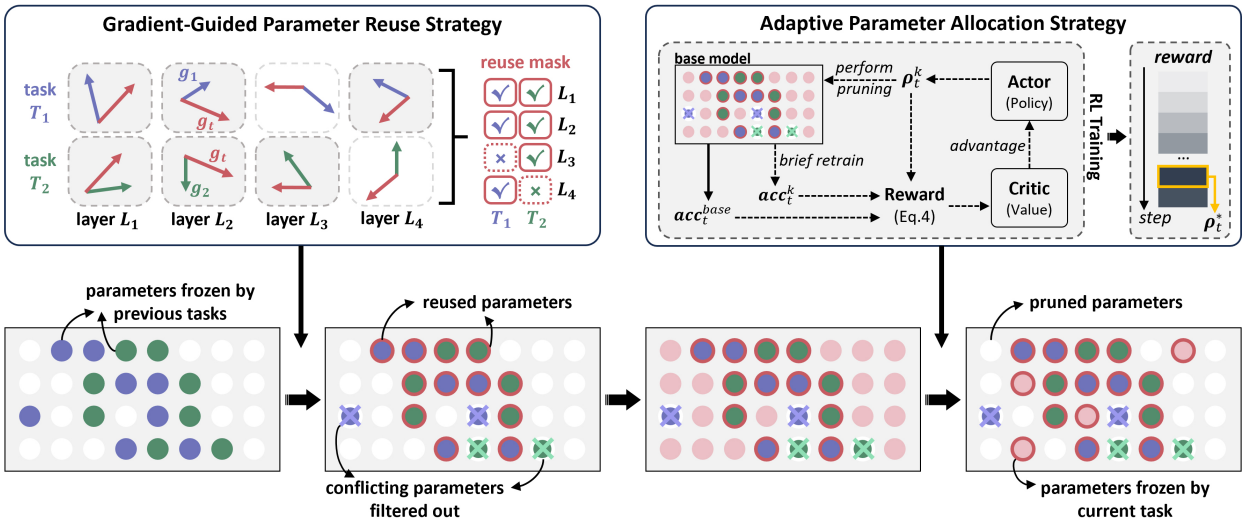


Figure 2. Overall framework of GRAPA. It first identifies reusable parameters for the current task from previously frozen ones based on gradient consistency, then uses A2C to adaptively allocate the minimal number of new parameters needed to sustain performance.

set of the previously frozen parameters for forward computation, which is referred to as reused parameters and denoted as $\psi_t \subseteq \bigcup_{i=1}^{t-1} \phi_i$. Accordingly, the set of parameters activated for task T_t is given by $\theta_t = \phi_t \cup \psi_t$.

Existing parameter isolation methods either rely solely on selecting reusable parameters via task-specific masks from a shared parameter space, or allocate exponentially decreasing amounts of new parameters to sequential tasks with a fixed ratio. However, they fail to consider the interplay between frozen parameter reuse and new parameter allocation. As a result, the dual objective of preserving the performance on previously learned tasks while efficiently acquiring new knowledge for upcoming tasks remains unsolved under constrained resources. The core challenge is, for each new task T_t, how to reuse a subset of compatible frozen parameters from previous tasks while allocating a minimal yet sufficient set of new parameters to support learning. The objective is to minimize the task-specific losses $\mathcal{L}_t(\cdot)$ across all tasks within a limited parameter budget B, formulated as

$$\min_{\theta_t} \sum_{t=1}^{N} \mathcal{L}_t(\theta_t), \quad \text{s.t.} \quad \left| \bigcup_{t=1}^{N} \theta_t \right| \leq B. \tag{1}$$

Addressing this challenge requires a mechanism that can: (i) selectively reuse frozen parameters compatible with the current task, and (ii) adaptively allocate new parameters based on task complexity and performance feedback.

4. Methodology

In resource-constrained continual learning, it is essential to balance task performance and parameter efficiency. To achieve this, we propose a two-stage framework named GRAPA, as illustrated in Figure 2. GRAPA first identifies and reuses a subset of previously frozen parameters whose gradient directions do not conflict with the current task, thereby maximizing knowledge reuse while avoiding harmful interference. Based on this, GRAPA allocates all remaining free parameters to the current task for training, and then applies a reinforcement-learning-based method to adaptively prune unnecessary parameters and freeze only necessary new parameters. This process aims to preserve task accuracy while minimizing the number of frozen parameters, thus improving long-term scalability. In the following, we provide a detailed explanation of the two core components of GRAPA, namely the gradient-guided parameter reuse strategy and the adaptive parameter allocation strategy. The pseudocode is provided in Algorithm 1.

4.1. Gradient-guided Parameter Reuse Strategy

The goal of reusing parameters from previous tasks is to enable knowledge transfer across tasks and thereby facilitate the learning of new tasks. In addition, we aim to reduce the reliance on newly allocated parameters by reusing non-conflicting frozen parameters from past tasks, while avoiding overly sparse model structures activated by the current task. However, if the gradient of reused parameters conflicts with the current task, it may lead to negative transfer or even catastrophic forgetting [28]. To efficiently reuse the frozen parameters while avoiding harmful interference, we adopt a gradient-consistency-based parameter reuse strategy. Specifically, let $\mathbf{g}_t^l$ denote the gradient from the current task T_t on layer l, and $\mathbf{g}_{t'}^l$ denote the gradient from a previous task $T_{t'}$ on the same layer. For each previous task, the gradients on its frozen parameters are stored at the convergence stage of training with a bounded and controllable storage overhead. When training a new task, we compute

Algorithm 1 Training Procedure of GRAPA

Input: Task sequence $\mathcal{T} = \{T_1, \ldots, T_N\}$, model θ
Parameter: Training epoch E, A2C steps K (≤ 20), accuracy threshold η, coefficients α, β, exploration rate ε
Output: Reuse and frozen parameter sets $\{(\psi_t, \phi_t)\}_{t=1}^{N}$

for each task T_t in $\mathcal{T}$ **do**
  // Stage 1: Gradient-guided Reuse
  if $t = 1$ **then**
    Set $\psi_t \leftarrow \emptyset$
  else
    for each epoch e **do**
      Compute T_t's gradient on frozen parameters
      for each layer l and each previous task $T_{t'}$ **do**
        Compute cosine similarity according to Eq.2
      end for
      Update reuse counter
    end for
    Obtain reuse set ψ_t for task T_t according to Eq.3
  end if
  // Stage 2: Adaptive Parameter Allocation
  Initialize trainable set $\theta_t^{free} = \theta \setminus \bigcup_{i<t} \phi_i$
  Train base model on T_t using ψ_t (frozen) and θ_t^{free}
  Evaluate baseline accuracy acc_t^{base}
  for $k = 1$ to K **do**
    Extract state s_t^k and select pruning rate ρ_t^k
    Prune model with rate ρ_t^k and briefly retrain
    Evaluate acc_t^k and compute reward r_t^k (Eq. 4)
    Update actor and critic networks via A2C
  end for
  Select ρ_t^* maximizing reward from experience
  Prune base model by ρ_t^*; freeze left weights ϕ_t
end for

the cosine similarity between its gradients and those of past tasks on the corresponding frozen parameters at each epoch

$$\cos(\mathbf{g}_t^l, \mathbf{g}_{t'}^l) = \frac{\mathbf{g}_t^l \cdot \mathbf{g}_{t'}^l}{\|\mathbf{g}_t^l\|_2 \, \|\mathbf{g}_{t'}^l\|_2}. \tag{2}$$

The gradients of two tasks are considered consistent on layer l if their cosine similarity exceeds 0. Parameters are reusable only when their gradients remain consistent for more than half of the total training epochs.

Accordingly, the set of parameters reused by task T_t is

$$\psi_t = \bigcup_{i=1}^{t-1} \bigcup_{l} \left\{ \phi_i^l \;\middle|\; \sum_{e=1}^{E} \mathbb{I}\big[\cos\big(\mathbf{g}_{t,e}^l, \mathbf{g}_i^l\big) > 0\big] > \frac{E}{2} \right\}, \tag{3}$$

where ϕ_i^l denotes the set of frozen parameters from task T_i on layer l. The reuse decision is recorded in a binary reuse mask M_t, where 1 indicates reuse and 0 indicates non-reuse.

4.2. Adaptive Parameter Allocation Strategy

In resource-constrained continual learning, it is crucial to control the number of parameters frozen for each task. Since these task-specific parameters are fixed after training, they cannot be further updated or reused by future tasks. Over-allocating new parameters will quickly exhaust the model's capacity and may lead to the failure of learning subsequent tasks, especially when transferable knowledge from earlier tasks is insufficient, such as in task sequences with increasing difficulty. Therefore, our objective is to minimize the number of parameters frozen by each task while maintaining its performance, i.e., to determine the minimal effective parameter subset required for each task. To this end, after completing full-parameter training on the current task, we apply pruning to remove redundant connections and retain only those parameters critical for learning the task, thereby constructing a parameter-efficient structure that meets the performance requirements.

Objective Clarification. The success of pruning depends on two key aspects: the choices of pruning criterion and pruning rate. Nowak et al. have demonstrated that, in highly sparse training regimes, simple magnitude-based pruning outperforms more complex heuristics in preserving model performance. Following this insight and further validation, we adopt parameter magnitude to measure parameter importance. The remaining challenge is to determine the maximal pruning rate that maintains acceptable performance on the current task. In practical task flows, heterogeneous tasks have different expected accuracy levels and parameter demands. As a result, manually tuning pruning rate without task-specific prior knowledge is impractical. To address this, we model pruning as a task-adaptive trial-and-error exploration problem and leverage the adaptability of reinforcement learning to unknown states. Specifically, we adopt Advantage Actor-Critic (A2C) algorithm [20], where the policy is updated via reward signals that guide the agent to balance pruning intensity and performance retention.

Training Process. Specifically, the agent consists of a policy network and a value network. The input state s_t^k at the k-th step of task T_t consists of five task-specific features, including two step-dependent variables (the pruning rate and accuracy from the previous step) and three task-descriptive attributes (number of classes, sample complexity, and separability). The task-descriptive states are computed based solely on the current task's data with negligible overhead and more details are in supplementary material. Before exploration begins, we assign all unfrozen parameters to the current task T_t to train a base model by setting the pruning rate to 0. Then we evaluate its accuracy on the current task acc_t^{base}. These values serve as the agent's initial step-dependent state. Then the full state vector is separately fed into the actor and critic networks. The actor network outputs the pruning rate $\rho_t^k \in (0, 1)$, while the critic network

estimates the state value $V(s_t^k)$. To encourage exploration, we adopt an ε-greedy strategy where the pruning rate is uniformly sampled from the interval $(0, 1)$ with probability $\varepsilon = 0.2$ and otherwise the pruning rate is taken from the actor's output. After each interaction, the pruning rate is applied to the base model to perform pruning, followed by a brief retraining to recover performance. The resulting accuracy acc_t^k of the pruned model is then used to compute the reward r_t^k. The reward function, which aims to balance pruning intensity and accuracy retention, is defined as

$$r_t^k = \begin{cases} -1, & \text{if } \delta_t^k < \eta \\ (\delta_t^k)^\alpha \cdot (\rho_t^k)^\beta, & \text{otherwise} \end{cases}, \tag{4}$$

where $\delta_t^k = \frac{acc_t^k}{acc_t^{base}}$ reflects the performance loss caused by pruning, and the threshold $\eta = 0.6$ provides a clear penalty signal when performance drops excessively, preventing futile exploration and promoting faster policy learning. The hyperparameters α and β respectively control the importance of model performance and parameter reduction, which are set to 5 and 1.5 respectively in this work. We provide sensitivity analysis of α and β in 5.2.5. This reward design encourages the agent to identify the maximal pruning rate that can be applied without causing significant degradation in model performance. The policy is optimized using policy-gradient updates, where the value function serves as a baseline to stabilize training and reduce variance.

Overhead Control. It is worth noting that although reinforcement learning typically introduces additional computational overhead due to its trial-and-error exploration, our agent is implemented as a lightweight model with only about 7K parameters, which is negligible compared to the 11.7M parameters of ResNet-18. In practice, training the agent accounts for less than 1.5% of the total time of a complete experiment. However, each exploration step requires a brief retraining phase to recover model performance. While each retraining is fast, too many exploration steps would lead to unnecessary overhead. Therefore, it is crucial to perform as few but as effective explorations as possible for each task. To further reduce the training cost and maintain policy robustness, we do not directly use the raw output of the policy network. Instead, we select the pruning rate with the highest reward among those satisfying the accuracy constraint from the agent's experience. In our experiments, each agent is trained for no more than 20 steps, making the additional overhead acceptable. This strategy effectively mitigates potential failures caused by fluctuations in policy output and further reduces the additional overhead introduced by the exploration process.

5. Experiments

We evaluate GRAPA following the standard continual learning evaluation method. When learning a new task, the training samples of previous tasks are inaccessible, ensuring privacy preservation and reducing data storage overhead in realistic deployment scenarios. After all tasks are trained, evaluation is conducted on their test sets using the corresponding classifiers. Therefore, the reported results include the forgetting of earlier tasks exhibited by each method.

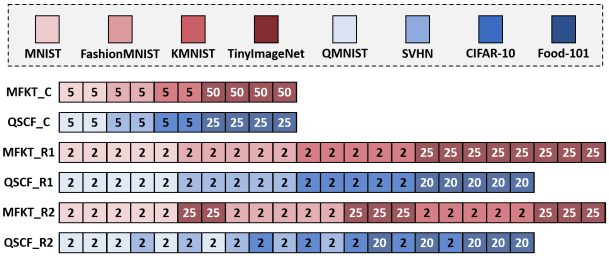


Figure 3. Composition of six continual task sequences. The numbers in the boxes denote the number of classes for each task.

5.1. Experimental Setup

Datasets and task sequences. To comprehensively evaluate the continual learning ability of GRAPA under diverse task structures and data distributions, we construct several task sequences based on eight datasets which are organized into two groups. The first dataset group **MFKT** consists of MNIST, FashionMNIST, KMNIST, and TinyImageNet, and the other group **QSCF** includes QMNIST, SVHN, CIFAR-10, and Food-101. Each group spans a wide range of visual domains, from simple handwritten digits to complex natural images, covering progressive increases in visual and semantic complexity. This diversity not only challenges the model's generalization ability across varying visual structures and category granularities but also provides a comprehensive benchmark for evaluating continual learning performance under limited model capacity. Then we split each dataset into several non-overlapping class groups to obtain task sequences. Figure 3 presents the task composition of all sequences used in our experiments. We evaluate GRAPA and the baseline methods on a lightweight CNN using MFKT_C and QSCF_C, and on ResNet-18 using MFKT_R1 and QSCF_R1. To further assess robustness to task order while preserving the progression of increasing difficulty, we interleave heterogeneous tasks to form MFKT_R2 and QSCF_R2, which are also evaluated on ResNet-18.

Baselines and Models. To evaluate our method, we compare it with representative baselines from different learning paradigms. *LwF* [16] is a regularization-based approach that mitigates forgetting through knowledge distillation. *PackNet* [18] belongs to the parameter isolation paradigm, which incrementally prunes and freezes task-specific weights to enable continual learning. *H2* [7] builds on PackNet by introducing Fisher information to selectively reuse parameters that are less important to previous tasks. *GrowBrain* [8] uses a hypernetwork to generate a set of pa-

	MNIST	F.MNIST	KMNIST	TinyImg.	Mean	QMNIST	SVHN	CIFAR	Food	Mean
Method	**Task sequence: MFKT_C, Model:CNN**					**Task sequence: QSCF_C, Model:CNN**				
Independent	99.49	95.19	95.78	47.09	76.86	99.36	88.44	83.52	56.21	77.34
LwF	27.34	37.79	40.11	37.34	34.86	26.25	38.92	44.97	50.61	41.86
PackNet	98.79	92.73	87.59	20.43	65.39	98.73	70.65	73.53	26.61	58.95
H2	99.53	95.35	96.70	32.17	71.78	99.51	92.80	84.46	24.11	64.81
GrowBrain	98.21	92.55	90.96	28.34	67.78	99.18	79.15	80.82	49.51	66.49
GRAPA	99.15	93.08	95.20	47.64	76.54	98.42	91.56	81.02	49.91	74.16
Method	**Task sequence: MFKT_R1, Model:ResNet-18**					**Task sequence: QSCF_R1, Model:ResNet-18**				
Independent	99.96	99.59	98.63	71.76	88.84	99.98	98.82	98.74	78.92	94.12
LwF	91.15	84.89	75.82	54.75	73.80	52.27	80.92	81.63	58.13	68.24
PackNet	99.82	99.29	97.01	30.25	74.90	99.76	97.87	88.99	33.97	80.64
H2	99.95	99.57	98.26	47.73	81.34	99.91	98.97	93.78	57.88	87.64
GrowBrain	99.79	99.38	98.49	48.97	81.74	99.76	90.62	88.70	48.84	81.98
GRAPA	99.80	99.03	97.70	61.60	85.89	99.66	97.64	93.80	69.89	90.25
Method	**Task sequence: MFKT_R2, Model:ResNet-18**					**Task sequence: QSCF_R2, Model:ResNet-18**				
Independent	99.96	99.59	98.63	71.76	88.84	99.98	98.82	98.74	78.92	94.12
LwF	50.32	83.45	92.18	27.82	58.80	52.11	77.18	70.38	41.91	60.39
PackNet	99.90	99.29	95.09	39.99	77.88	99.89	98.01	91.87	33.36	80.78
H2	99.75	99.16	96.03	49.17	81.22	99.98	98.86	97.01	35.05	82.72
GrowBrain	99.80	99.35	98.53	49.06	81.78	99.77	90.11	88.39	48.91	81.79
GRAPA	96.50	95.80	97.35	64.09	85.26	99.83	98.12	92.10	62.08	88.03

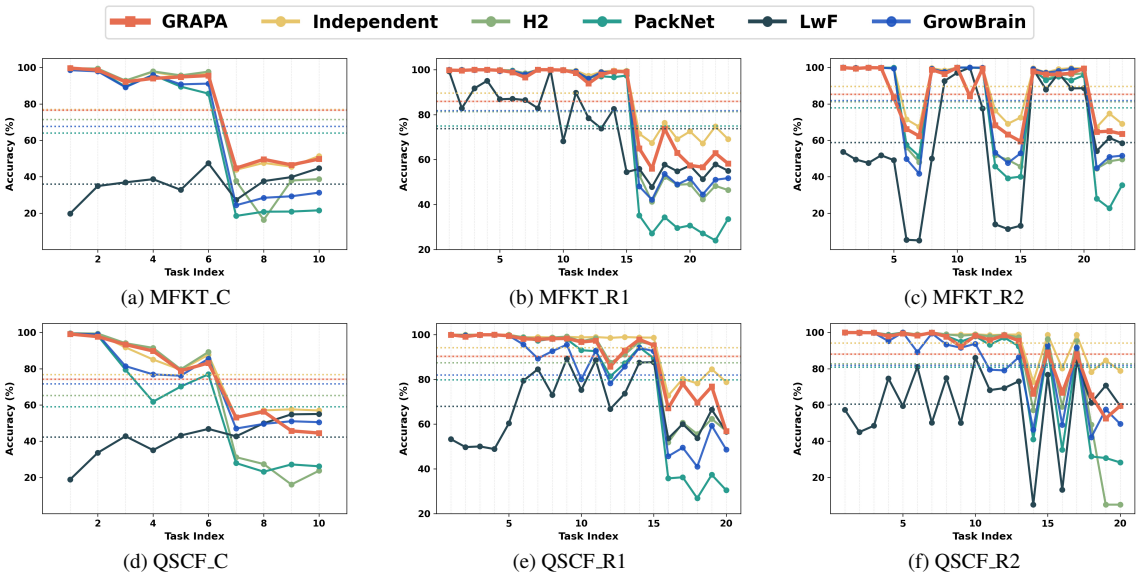
Table 1. Average test accuracy (%) of different methods on each dataset within every task sequence. The baseline Independent serves as an empirical upper bound, as it is trained without model capacity constraints or inter-task interference.

rameters that selectively adjust the base network for new tasks. We also include the *Independent* baseline, which trains a separate full model for each task. Since it avoids both task interference and plasticity issues, it serves as a performance upper bound for each task (though not a strict optimum). In addition, we evaluate two ablation variants of GRAPA. *GRAPA (w/o GR)* replaces our gradient-guided parameter reuse strategy with a naive reuse of all previously frozen parameters. *GRAPA (w/o APA)* uses a fixed pruning rate instead of our adaptive allocation strategy. Since we focus on resource-constrained scenarios, we employ a lightweight CNN and ResNet-18 for evaluation. Both architectures use a shared backbone for extracting features and task-specific classification heads.

5.2. Results and Analysis

5.2.1. Overall Performance

Table 1 presents the average test accuracy of different continual learning methods on each dataset, as well as the overall average accuracy across each sequence. Across all task sequences, LwF achieves significantly lower final test accuracy on early tasks compared to other baselines, and in several sequences, it completely forgets the tasks from the first dataset, despite performing close to Independent during training. In contrast, the parameter-isolation methods PackNet and H2 preserve previously learned knowledge by freezing task-specific parameters after training. Experimental results show that these methods perform similarly to the Independent baseline on early tasks, which demonstrates their strong stability. However, LwF performs better on later tasks. In the regularized task sequences (MFKT_C, QSCF_C, MFKT_R1, and QSCF_R1), LwF achieves higher performance on later, more complex tasks than all existing parameter-isolation methods. In contrast, PackNet and H2 perform poorly on later tasks because they rely on a fixed pruning rate, which rapidly depletes model capacity during continual training. This confirms that regularization-based methods are less effective at preventing forgetting, while existing parameter-isolation methods, although solving forgetting effectively, suffer from a loss of plasticity. Although GrowBrain introduces a hypernetwork-based transformation mechanism to old parameters on top of the traditional parameter-isolation framework, its success relies on strong inter-task correlations, showing limited generalization ability on heterogeneous task sequences with progressively increasing difficulty. In comparison, our method GRAPA achieves consistently balanced performance across all tasks sequences, outperforming all baselines under both CNN and ResNet-18 models. Because GRAPA freezes parameters from early tasks, it maintains near-upper-bound performance on earlier datasets. At the same time, by freezing only a minimal set of necessary parameters for early tasks, GRAPA retains strong plasticity for later, more complex tasks. These results demonstrate that our method effec-



(a) MFKT_C (b) MFKT_R1 (c) MFKT_R2

(d) QSCF_C (e) QSCF_R1 (f) QSCF_R2

Figure 4. Final test accuracy of each method on every task across the six sequences. The dashed lines represent the average accuracy of each method over its entire task sequence, providing a summary of overall performance.

tively balances stability and plasticity in continual learning.

5.2.2. Performance on Specific Tasks

Figure 4 shows the final test accuracy of each method on every task across all the sequences. In sequence MFKT_R1 (Figure 4b), the parameter-isolation method PackNet maintains high accuracy on the first 15 relatively simple tasks. Its performance is close to the Independent baseline and significantly better than the regularization-based method LwF. However, it applies a fixed pruning rate. As tasks progress, the number of available new parameters for new tasks drops quickly. Starting from task 16, the difficulty increases significantly. With limited new parameters and ineffective knowledge from previous tasks, the performance of PackNet drops sharply. H2 and GrowBrain additionally introduce mechanisms for knowledge reuse and parameter transformation, which enhance performance to some extent. However, they both remain constrained by the limited plasticity imposed by the fixed pruning rate, resulting in suboptimal accuracy on the last 8 TinyImageNet tasks. Similar trend is also observed in sequence QSCF_R1 (Figure 4e) and in the results on the CNN models (Figures 4a and 4d). These results highlight the strength of parameter isolation in addressing forgetting but also reveal its lack of flexibility when learning complex tasks.

Sequences MFKT_R2 and QSCF_R2 further reveal the sensitivity of each method to task order. In sequence MFKT_R2 (Figure 4c), TinyImageNet is divided into three parts and inserted at different positions in the task sequence (task 6–7, 13–15, and 21–23). On these tasks, LwF shows a clear upward trend in performance, while PackNet continues to decline. In sequence QSCF_R2 (Figure 4f), CIFAR-10 is interleaved between the Food-101 tasks. As a result, LwF exhibits forgetting on the first two Food-101 tasks (task 14, 16), while PackNet and H2 show performance decline across these Food-101 tasks (task 14, 16, 18-20). Such results suggest that although LwF struggles to retain early knowledge, it adapts more flexibly to complex new tasks. PackNet and H2 preserve previous knowledge more effectively but lack the plasticity needed to handle new tasks. In contrast, GRAPA and GrowBrain achieve more robust performance across these sequences. In GRAPA, when tasks vary in difficulty and arrive in mixed order, it adaptively allocates parameters based on each task's needs. This strategy helps the model overcome capacity bottlenecks and maintain a better balance between stability and plasticity. As a result, it preserves accuracy on early tasks and significantly improves performance on later, more challenging ones.

Method	CNN		ResNet-18			
	M._C	Q._C	M._R1	M._R2	Q._R1	Q._R2
GRAPA(w/o GR)	74.55	72.86	80.30	62.79	81.29	85.57
GRAPA(w/o APA)	73.73	70.07	73.04	75.93	71.00	78.84
GRAPA	76.54	74.16	85.89	85.26	90.25	88.03

Table 2. Mean test accuracy (%) of ablations on each sequence. MFKT and QSCF are abbreviated as M. and Q. respectively here.

5.2.3. Ablation Study

Table 2 shows the mean test accuracy of ablation variants and full GRAPA across six task sequences on the lightweight CNN and ResNet-18 models. Removing either the gradient-guided reuse (GR) or adaptive parameter allocation (APA) component leads to performance degradation, confirming both are essential. GRAPA (w/o GR)

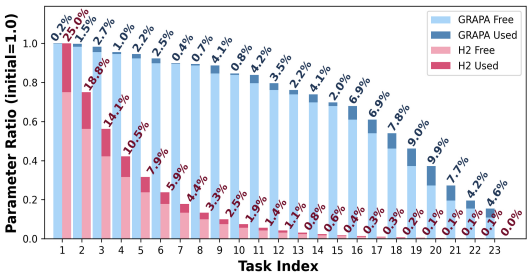
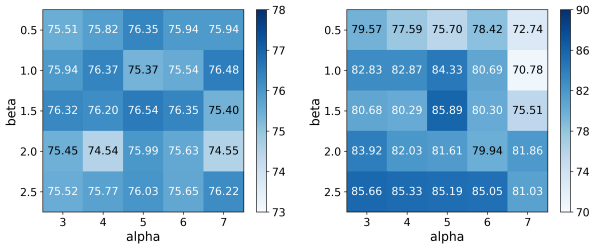


Figure 5. Parameter allocation comparison between GRAPA and H2 on task sequence MFKT_R1.



(a) CNN (b) ResNet-18

Figure 6. Sensitivity analysis of GRAPA with respect to the hyperparameters α and β, which control the trade-off between accuracy preservation and parameter reduction.

reuses all frozen parameters without considering gradient conflicts, thus introduces negative interference from incompatible tasks. GRAPA (w/o APA) employs a fixed pruning rate, which rapidly freezes a large portion of parameters, thereby limiting the model's plasticity, especially when handling longer task sequences. The degradation mainly results from failures on subsequent complex tasks. For instance, in sequence MFKT_R1 and QSCF_R1, GRAPA (w/o APA) fails on TinyImageNet (25.11%) and Food-101 (24.25%).

We further compare the RL-based pruning strategy with two alternatives in sequence QSCF_R1. The first uses a linearly decreasing pruning rate starting from 90% and reducing by 2% per task, while the second applies a binary-search-based strategy. These two strategies achieve average accuracies of 79.58% and 83.04%, both lower than GRAPA's 90.25%. Although the linear-decay strategy introduces less overhead since it requires no exploration, it relies on prior knowledge to set hyperparameters and is therefore impractical for unknown task streams. In contrast, the RL-based strategy consistently outperforms these heuristic strategies and is more robust to variations in task sequences.

5.2.4. Parameter Freezing and Capacity Retention

We study how different methods allocate parameters and retain model capacity as tasks arrive sequentially. Figure 5 shows a comparison between GRAPA and the parameter-isolation method H2, which adopts a fixed pruning rate, on the sequence MFKT_R1. Because H2 allocates the same proportion of parameters to each task based on the remaining unfrozen ones, the number of parameters available for new tasks decreases exponentially as training progresses. If the model fails to learn transferable representations from early tasks, its performance on subsequent tasks will quickly decrease. By the time the TinyImageNet tasks arrive, less than 1.34% of the parameters in H2 remain trainable, severely constraining model plasticity. In contrast, GRAPA adaptively freezes only the minimal subset of parameters required to maintain task performance, thereby preserving more available parameters for later, more complex tasks. This strategy inherits the no-forgetting advantage of parameter isolation while maximizing model plasticity across heterogeneous task sequences.

5.2.5. Sensitivity Analysis

We analyze the sensitivity of GRAPA to the hyperparameters in the reward function α and β using MFKT_C (CNN) and MFKT_R1 (ResNet-18). Specifically, α is varied in $3, 4, 5, 6, 7$ and β in $0.5, 1.0, 1.5, 2.0, 2.5$. Each study includes 25 combinations and is visualized as heatmaps in Figure 6. The results indicate that GRAPA is insensitive to these hyperparameters on the CNN model, where the performance gap between the best and worst combinations is within 2%. In contrast, more pronounced variations are observed on ResNet-18. When α is large and β is small, GRAPA's performance declines noticeably. In such cases, the reward function overemphasizes task performance while underweighting parameter efficiency, prompting early tasks to maximize accuracy at the expense of rapidly freezing most of the model's capacity. Consequently, later complex tasks lack sufficient unfrozen parameters, leading to degraded performance. Based on this observation and empirical evidence, we fix $\alpha = 5$ and $\beta = 1.5$ as the default configuration for all experiments. This setting achieves a stable balance between performance retention and parameter efficiency across different task sequences and models.

6. Discussion

Although existing parameter isolation methods are effective in preventing forgetting, they often overlook the challenge of maintaining model plasticity under limited resources. GRAPA addresses this issue by selecting non-conflicting frozen parameters from previous tasks and allocating only the minimal number of new parameters needed to maintain task performance. It demonstrates superior stability and plasticity across multiple task sequences with increasing complexity. While GRAPA substantially improves parameter efficiency, the total number of tasks that can be supported is still inherently bounded by the model's capacity. Therefore, building on this study, future work can explore intentional forgetting mechanisms that identify and discard outdated tasks or less important parameters over time, thereby achieving a more practical balance between stability and plasticity in on-device continual learning.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grants U25A20437, 62372473, and 92270204, in part by the CAS Project for Young Scientists in Basic Research under Grant YSBR-118. The authors are grateful for resources from the High Performance Computing Center of Central South University. Shigeng Zhang and Xuan Liu are the corresponding authors of this paper.

References

[1] Rahaf Aljundi, Francesca Babiloni, Mohamed Elhoseiny, Marcus Rohrbach, and Tinne Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European conference on computer vision (ECCV)*, pages 139–154, 2018. 2

[2] Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. Gradient based sample selection for online continual learning. *Advances in neural information processing systems*, 32, 2019. 2

[3] Jihwan Bang, Heesu Kim, YoungJoon Yoo, Jung-Woo Ha, and Jonghyun Choi. Rainbow memory: Continual learning with a memory of diverse samples. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8218–8227, 2021.

[4] Zalán Borsos, Mojmir Mutny, and Andreas Krause. Coresets via bilevel optimization for continual learning and streaming. *Advances in neural information processing systems*, 33: 14879–14890, 2020. 2

[5] Rui Gao and Weiwei Liu. Ddgr: Continual learning with deep diffusion-based generative replay. In *International Conference on Machine Learning*, pages 10744–10763. PMLR, 2023. 2

[6] Ahmet Iscen, Jeffrey Zhang, Svetlana Lazebnik, and Cordelia Schmid. Memory-efficient incremental learning through feature adaptation. In *European conference on computer vision*, pages 699–715. Springer, 2020. 2

[7] Hyundong Jin and Eunwoo Kim. Helpful or harmful: Inter-task association in continual learning. In *European conference on computer vision*, pages 519–535. Springer, 2022. 1, 2, 5

[8] Hyundong Jin, Gyeong-hyeon Kim, Chanho Ahn, and Eunwoo Kim. Growing a brain with sparsity-inducing generation for continual learning. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 18961–18970, 2023. 1, 2, 5

[9] Sangwon Jung, Hongjoon Ahn, Sungmin Cha, and Taesup Moon. Continual learning with node-importance based adaptive group sparse regularization. *Advances in neural information processing systems*, 33:3647–3658, 2020. 1

[10] Haeyong Kang, Rusty John Lloyd Mina, Sultan Rizky Hikmawan Madjid, Jaehong Yoon, Mark Hasegawa-Johnson, Sung Ju Hwang, and Chang D Yoo. Forget-free continual learning with winning subnetworks. In *International conference on machine learning*, pages 10734–10750. PMLR, 2022. 1, 2

[11] Sanyam Kapoor, Theofanis Karaletsos, and Thang D Bui. Variational auto-regressive gaussian processes for continual learning. In *International Conference on Machine Learning*, pages 5290–5300. PMLR, 2021. 1

[12] Zixuan Ke, Bing Liu, and Xingchang Huang. Continual learning of a mixed sequence of similar and dissimilar tasks. *Advances in neural information processing systems*, 33:18493–18504, 2020. 1

[13] Sanghwan Kim, Lorenzo Noci, Antonio Orvieto, and Thomas Hofmann. Achieving a better stability-plasticity trade-off via auxiliary networks in continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11930–11939, 2023. 1

[14] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017. 1, 2

[15] Jiyong Li, Dilshod Azizov, Shangsong Liang, et al. Contrastive continual learning with importance sampling and prototype-instance relation distillation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 13554–13562, 2024. 1

[16] Zhizhong Li and Derek Hoiem. Learning without forgetting. In *Proceedings of the European conference on computer vision (ECCV)*, pages 614–629, 2016. 2, 5

[17] Yan-Shuo Liang and Wu-Jun Li. Inflora: Interference-free low-rank adaptation for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 23638–23647, 2024. 2

[18] Arun Mallya and Svetlana Lazebnik. Packnet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 7765–7773, 2018. 1, 2, 5

[19] Arun Mallya, Dillon Davis, and Svetlana Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *Proceedings of the European conference on computer vision (ECCV)*, pages 67–82, 2018. 1, 2

[20] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016*, pages 1928–1937, 2016. 4

[21] Aleksandra Nowak, Bram Grooten, Decebal Constantin Mocanu, and Jacek Tabor. Fantastic weights and how to find them: Where to prune in dynamic sparse training. *Advances in Neural Information Processing Systems*, 36:55160–55192, 2023. 4

[22] Grégoire Petit, Adrian Popescu, Hugo Schindler, David Picard, and Bertrand Delezoide. Fetril: Feature translation for exemplar-free class-incremental learning. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 3911–3920, 2023. 2

[23] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier

and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pages 2001–2010, 2017. 1, 2

[24] Jonathan Schwarz, Wojciech Czarnecki, Jelena Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. Progress & compress: A scalable framework for continual learning. In *International conference on machine learning*, pages 4528–4537. PMLR, 2018. 1

[25] Rishabh Tiwari, Krishnateja Killamsetty, Rishabh Iyer, and Pradeep Shenoy. Gcr: Gradient coreset based replay buffer selection for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 99–108, 2022. 2

[26] Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. Supermasks in superposition. *Advances in neural information processing systems*, 33:15173–15184, 2020. 1, 2

[27] Yichen Wu, Hongming Piao, Long-Kai Huang, Renzhen Wang, Wanhua Li, Hanspeter Pfister, Deyu Meng, Kede Ma, and Ying Wei. SD-loRA: Scalable decoupled low-rank adaptation for class incremental learning. In *The Thirteenth International Conference on Learning Representations*, 2025. 2

[28] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in neural information processing systems*, 33:5824–5836, 2020. 3

[29] Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International conference on machine learning*, pages 3987–3995. PMLR, 2017. 2

[30] Da-Wei Zhou, Hai-Long Sun, Jingyi Ning, Han-Jia Ye, and De-Chuan Zhan. Continual learning with pre-trained models: A survey. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, pages 8363–8371, 2024. 2